Efficient Kernel UCB for Contextual Bandits

AISTATS 2022

Houssam Zenati [1, 2] Alberto Bietti[3] Eustache Diemert [1 3]
Julien Mairal [2] Matthieu Martin [1] Pierre Gaillard [2]

[1]Criteo AI Lab

[2]Univ. Grenoble Alpes, Inria

[3]Center for Data Science, New York University

January 14, 2022

Sequential learning with bandit feedback

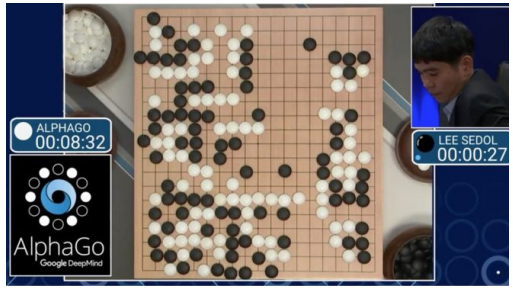
Problem: In interactive systems, full information is not always available.





We wish to learn to **take actions** so as to **maximize a reward** signal...

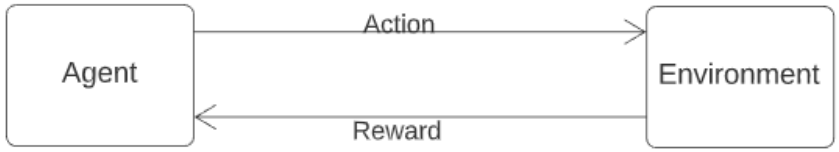




... while **ensuring efficient learning** for large-scale real-world applications.

What are contextual bandits?

At each round t, a bandit *agent* takes an *action* a_t and receives a *reward* r_t from the environment. The *agent* learns to take *actions* so as to maximise the *reward* function r.



In the contextual setting, a *contextual* information x_t is provided by the *environment* and included in the *reward* function $r := r(x, a)$. To evaluate the performance of a contextual bandit agent, we use the *regret*

$$R_T := \mathbb{E}\left[\sum_{t=1}^{T} \max_{a \in \mathcal{A}} r(x_t, a) - \sum_{t=1}^{T} r_t\right]. \tag{1}$$

Kernel UCB setting

Context $x_t \in \mathcal{X}$, actions $a_t \in \mathcal{A}$ and associated reward

$$r_t = \langle \theta^*, \phi(x_t, a_t) \rangle + \varepsilon_t \,,$$

where θ^* is an optimal parameter in the RKHS. And ϕ is a feature map associated to a kernel k.

The algorithm estimates the parameter θ^* with a regularized least squares:

$$\hat{\theta}_t \in \arg\min_{\theta \in \mathcal{H}} \left\{ \sum_{s=1}^{t} \left(\langle \theta, \phi(x_s, a_s) \rangle - r_s \right)^2 + \lambda \|\theta\|^2 \right\}. \tag{2}$$

Effective dimension in kernel methods

Even though the RKHS may be infinite dimensional, the kernel matrix $K_T = [k(s_i, s_j)]_{1 \leq i,j \leq T}$ has a most T eigenvalues. The data therefore lives in a space of dimension at most T.

Intuition: Some data may be correlated, so this dimension can be much smaller. The effective dimension of the problem is then defined as

$$d_{\text{eff}}(\lambda, T) := \text{Tr}(K_T(K_T + \lambda I_T)^{-1}) \,. \tag{3}$$

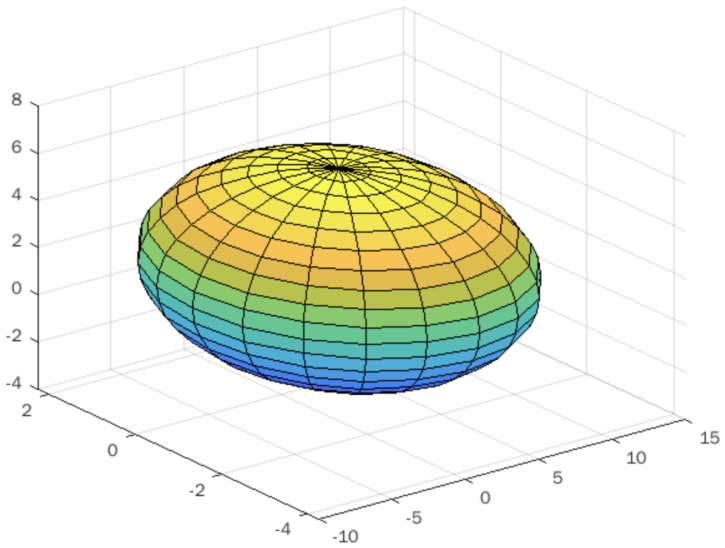
$d_{\text{eff}}(\lambda, T)$ approximately counts the number of eigenvalues bigger than a parameter $\lambda > 0$.

UCB: Upper Confidence Bound

UCB algorithms build confidence bounds around the estimate $\hat{\theta}$. With an ellipsoidal confidence set

$$\mathcal{C}_t = \{\theta \in \mathcal{H} : \|\theta - \hat{\theta}_{t-1}\|_{V_{t-1}} \leq \beta\}.$$

where V_t is a regularized covariance matrix depending on the data.



Lemma

Defining $\beta \approx \sqrt{\lambda}\|\theta^*\| + \sqrt{d_{\text{eff}}}$. *Then, with high probability, for all* $t \in [T]$

$$\left\|\hat{\theta}_t - \theta^*\right\|_{V_t} \leq \beta. \tag{4}$$

The true parameter θ^* *belongs to the confidence set* $\mathcal{C}_t$

Kernel UCB rule

As previously mentioned, for each action $a \in \mathcal{A}$, the algorithm takes the upper bound on the confidence set C_t based on previous samples:

$$\text{K-UCB}_t(a) = \max_{\theta \in \mathcal{C}_t} \langle \theta, \phi(x_t, a) \rangle \tag{5}$$

With $\mathcal{C}_t$ in that ellispoid form, we can write the solution of Eq. (5) as

$$\text{K-UCB}_t(a) = \underbrace{\langle \hat{\theta}_{t-1}, \phi(x_t, a) \rangle}_{\hat{\mu}_{t,a}} + \beta_t(\delta)^{1/2} \underbrace{\|\phi(x_t, a)\|_{V_{t-1}^{-1}}}_{\hat{\sigma}_{t,a}}. \tag{6}$$

At time t, the algorithm then selects optimistically:

$$a_t \in \arg\max_{a \in \mathcal{A}} \text{K-UCB}_t(a). \tag{7}$$

Algorithm

Proposition

The algorithm enjoys the following pseudo-regret bound:

$$R_T \lesssim \sqrt{T}\left(\|\theta^*\|\sqrt{\lambda d_{\text{eff}}} + d_{\text{eff}}\right),$$

under the capacity condition $d_{\text{eff}} \leq (T/\lambda)^{\alpha}$ *for* $0 \leq \alpha \leq 1$ *, the regret of K-UCB is bounded as*

$$R_T \lesssim T^{\frac{1+3\alpha}{2+2\alpha}}$$

At each round t, the algorithm estimates the mean $\hat{\mu}_{t,a}$ and standard deviation $\hat{\sigma}_{t,a}$ of the confidence bounds, for each action $a \in \mathcal{A}$.

Problem: The algorithm runs in $\mathcal{O}(T^2)$ space and $\mathcal{O}(T^3)$ time!

Contributions

In this work we present the following contributions:

- **K-UCB analysis:** we extend the OFUL (Abbasi-yadkori, Pál, and Szepesvári, 2011) analysis with non-trivial adaptations to handle both infinite dimension and kernel approximations
- **EK-UCB analysis:** we analyze the projected EK-UCB algorithm to achieve the best time-complexity and leverage the KORS algorithm (Calandriello, Lazaric, and Valko, 2017) to obtain an incremental structure of the projections
- **KORS regret/complexity compromise:** unlike previous works, we explicit the regret-complexity trade-off under the capacity condition assumption

Idea: Approximations of the RKHS

Nyström approximations find the orthogonal projection of a feature map φ onto a subspace $\mathcal{F}$. $\psi(x) = \arg\min_{f \in \mathcal{F}} \|\varphi(x) - f\|_{\mathcal{H}}$

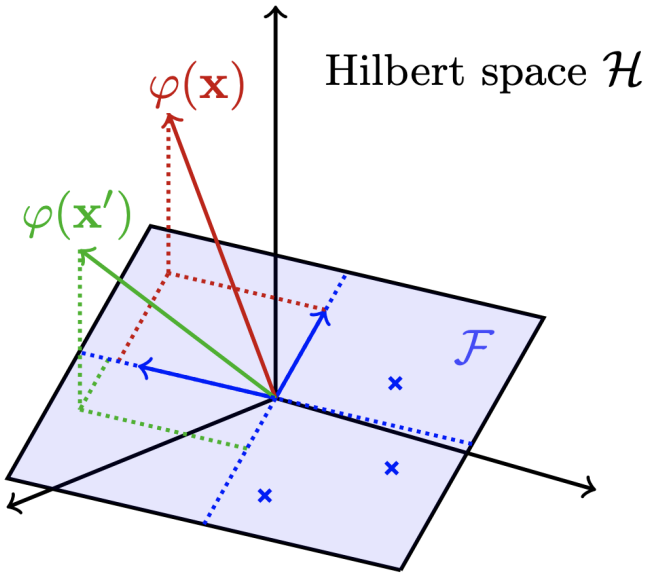


Figure: Nyström approximation of a feature map φ in the RKHS $\mathcal{H}$

Projected Ellipsoid

Now, we estimate the center of the ellipsoid in the projected $\tilde{\mathcal{H}}_t$ to build the confidence set $\tilde{\mathcal{C}}_t := \left\{\theta \in \mathcal{H} : \|\theta - \tilde{\theta}_{t-1}\|_{\tilde{V}_{t-1}} \leq \tilde{\beta}\right\}$ where $\tilde{V}_t$ is a projected regularized covariance matrix depending on the data.

Lemma

Defining $\tilde{\beta}_{t+1}(\delta) \approx \left(\sqrt{\lambda} + \sqrt{\mu_t}\right)\|\theta^*\| + \sqrt{d_{\text{eff}}}$

where $\mu_t := \|(I - P_t)F_t^{1/2}\|^2$. *Then, with high probability, for all* $t \in [T]$

$$\left\|\tilde{\theta}_t - \theta^*\right\|_{\tilde{V}_t} \leq \tilde{\beta}_{t+1}(\delta). \tag{8}$$

The center of the ellipsoid is now in the projected space.

Dictionary Building - KORS subroutine

Use an incremental Nyström dictionary $\mathcal{Z}_t$ for $\tilde{\mathcal{H}}_t = \text{Span}\{\phi(s), s \in \mathcal{Z}_t\}$.
The KORS algorithm:

- Compute leverage scores $\tilde{\tau}_t$ from the previous dictionary $\mathcal{Z}$, the state s_t, the KORS parameter μ
- Compute a probability $\tilde{p}_t$ from the leverage score $\tilde{\tau}_t$
- Draw a sample from Bernouilli $\mathcal{B}(\tilde{p}_t)$ to decide whether to include s_t to the dictionary $\mathcal{Z}$

Proposition

For $\delta > 0$, the sequence of dictionaries $\mathcal{Z}_1 \subset \mathcal{Z}_2 \subset \ldots \mathcal{Z}_T$ learned by KORS with parameters $\mu > 0$ satisfies with probability $1 - \delta$, $\forall t \geq 1$

$$\|(I - P_t)F_t^{1/2}\|^2 \leq \mu \quad \textit{and} \quad |\mathcal{Z}_t| \approx d_{\text{eff}}(\mu, T)\,.$$

EK-UCB regret bound

Theorem

Let $T \geq 1$ and $\theta^* \in \mathcal{H}$. Then, the EK-UCB algorithm with regularization λ along with KORS updates with parameter μ satisfies the regret bound

$$R_T \lesssim \sqrt{T}\left(\sqrt{\frac{\mu m}{\lambda}}+\sqrt{d_{\text{eff}}}\right)\left(\|\theta^*\|(\sqrt{\lambda}+\sqrt{\mu})+\sqrt{d_{\text{eff}}}\right),$$

where $m := |\mathcal{Z}_T|$. In particular, the choice $\mu = \lambda$ yields $m \lesssim d_{\text{eff}}$ and the bound

$$R_T \lesssim \sqrt{T}\left(\|\theta^*\|\sqrt{\lambda d_{\text{eff}}} + d_{\text{eff}}\right).$$

Furthermore, the algorithm runs in $O(Tm)$ space complexity and $O(CTm^2)$ time complexity.

Regret/Complexity Compromise

The following corollary exhibits the regret/complexity compromise under which the algorithm recovers the original regret rate of K-UCB.

Corollary

Assuming the capacity condition $d_{\text{eff}} \leq (T/\lambda)^{\alpha}$ *for* $0 \leq \alpha \leq 1$ *. Let* $m \geq 1$*, under the assumptions of Thm. 3, the regret of EK-UCB satisfies*

$$R_T \lesssim \begin{cases} Tm^{\frac{\alpha-1}{2\alpha}} & \text{if } m \leq T^{\frac{\alpha}{1+\alpha}} \\ T^{\frac{1+3\alpha}{2+2\alpha}} & \text{otherwise} \end{cases}$$

for the choice $\lambda = \mu = Tm^{-1/\alpha}$*. Furthermore, the algorithm runs in* $O(Tm)$ *space complexity and* $O(CTm^2)$ *time complexity.*

We can either recover the original regret rate or suffer worse regret if m is too small.

Related Work

Algorithm	Regret	Space	Time Complexity
CGP-UCB (Krause and Ong, 2011)	$\mathcal{O}(\sqrt{T}d_{\text{eff}})$	$\mathcal{O}(T^2)$	$\mathcal{O}(T^3)$
SupKernelUCB (Valko et al., 2013)	$\mathcal{O}(\sqrt{Td_{\text{eff}}\log(N)})$	$\mathcal{O}(T^2)$	$\mathcal{O}(T^3)$
K-UCB (ours)	$\mathcal{O}(\sqrt{T}(\sqrt{\lambda d_{\text{eff}}}+d_{\text{eff}})$	$\mathcal{O}(T^2)$	$\mathcal{O}(T^3)$
C-BKB (Calandriello, Carratino, et al., 2019)	$\mathcal{O}(\sqrt{T}(\sqrt{\lambda d_{\text{eff}}}+d_{\text{eff}})$	$\mathcal{O}\left(Td_{\text{eff}}\right)$	$\mathcal{O}\left(T^2d_{\text{eff}}^2+Td_{\text{eff}}^2\right)$
C-BBKB (Calandriello, Carratino, et al., 2020)	$\mathcal{O}(\sqrt{T}d_{\text{eff}})$	$\mathcal{O}\left(Td_{\text{eff}}\right)$	$\mathcal{O}\left(Td_{\text{eff}}^3+Td_{\text{eff}}^2\right)$
EK-UCB (ours)	$\mathcal{O}(\sqrt{T}(\sqrt{\lambda d_{\text{eff}}}+d_{\text{eff}})$	$\mathcal{O}\left(Td_{\text{eff}}\right)$	$\mathcal{O}(Td_{\text{eff}}^2)$

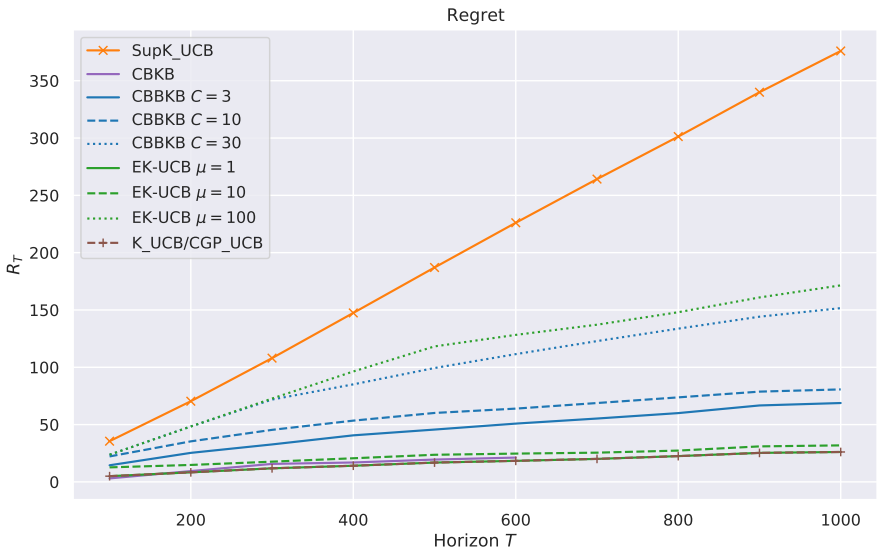
Table: Comparison of regrets, space and time complexities

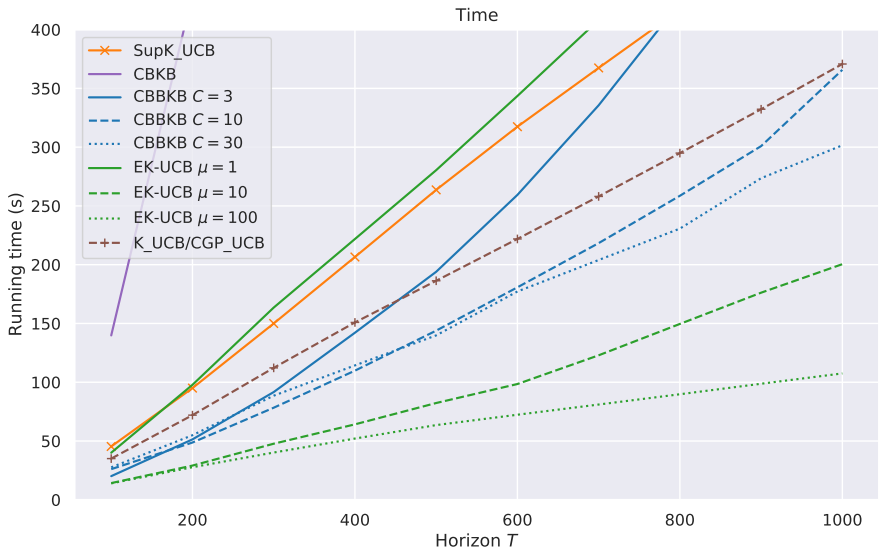
Our algorithm enjoys the best time complexity while achieving the same regret rates as original UCB methods.

Numerical Experiments

Experimental setup. We consider synthetic environments with contexts uniformly distributed in $[0,1]^p$, and actions in $[0,1]$. The reward is of the form $r(x,a) = \max(0, 1 - \|a - a^*\|_1 - \langle w^*, x - x^* \rangle)$.

Baselines. In our experiments, we to K-UCB, SupK-UCB and to works which focus on improving the $\mathcal{O}(T^3)$ time-complexity as CBKB and CBBKB.





Experiments

In this example, for $T = 1000$, we have set $\lambda = 10$ (of the order of $\sqrt{T}$):

- The value of $\mu = \lambda$ indeed achieves a good tradeoff between regret and time.
- For smaller μ, the algorithm achieves a better regret but pays a higher time complexity. Similarly smaller C induce a more frequent dictionary update.
- CGP-UCB/K-UCB obtain best regret
- EK-UCB ($\mu = 1$), CBKB (which is CBBKB with $C = 1$) essentially take the full dictionary $m \approx T$ and thus also match K-UCB, but with dictionary building computational overheads.
- SupK-UCB performs poorly due to its over-exploring elimination strategy, even though it has a tighter regret.

Take Home Message

- The EK-UCB algorithm runs in $\mathcal{O}(Td_{\text{eff}})$ space and $\mathcal{O}(CTd_{\text{eff}}^2)$ time complexity, which significantly improves over the standard contextual kernel UCB method.
- The incremental projection updates are crucial to perform efficient approximations in the joint context-action space
- A natural question is whether we may obtain algorithms with better regret guarantees similar to Valko et al. (2013) in the finite action case, while also achieving gains in computational efficiency as in our work.

References

- Abbasi-yadkori, Yasin, Dávid Pál, and Csaba Szepesvári (2011). "Improved Algorithms for Linear Stochastic Bandits". In: *Adv. Neural Information Processing Systems (NIPS)*.
- Calandriello, Daniele, Luigi Carratino, et al. (2019). "Gaussian Process Optimization with Adaptive Sketching: Scalable and No Regret". In: *Conference on Learning Theory (COLT)*.
- – (2020). "Near-linear time Gaussian process optimization with adaptive batching and resparsification". In: *International Conference on Machine Learning (ICML)*.
- Calandriello, Daniele, Alessandro Lazaric, and Michal Valko (2017). "Efficient Second-Order Online Kernel Learning with Adaptive Embedding". In: *Adv. Neural Information Processing Systems (NIPS)*.
- Krause, Andreas and Cheng S Ong (2011). "Contextual Gaussian process bandit optimization". In: *Adv. Neural Information Processing Systems (NIPS)*.
- Valko, Michal et al. (2013). "Finite-Time Analysis of Kernelised Contextual Bandits". In: *Conference on Uncertainty in Artificial Intelligence (UAI)*.